Conversation Intelligence System

Interview question bank with model answers

Questions	95
Sections	12
Difficulty	25 easy / 48 medium / 22 hard
Covers	AMI data, Deepgram Nova-3, pyannote community-1, overlap handling, structured extraction, WER / cpWER / DER, LLM judges and calibration, LiveKit real time and latency, ElevenLabs, deployment and Arize

How to use this. Cover the answer and say yours out loud first, then compare. Model answers are written to be spoken in 30-90 seconds. Follow-ups are what a good interviewer asks next. 'In the repo' points to the code that backs the answer, so you can cite it. Numbers quoted from vendors (such as the pyannote model card) are labelled; everything else is a design target until you run the evals.

Contents

1. Project overview and framing . . . 3
2. Data: the AMI Meeting Corpus . . . 4
3. Transcription with Deepgram Nova-3 . . . 5
4. Diarization and overlapping speech . . . 7
5. Alignment and the structured record . . . 9
6. LLM extraction and grounding . . . 10
7. Speech metrics: WER, cpWER, DER . . . 12
8. Evaluating summaries and calibrating the LLM judge . . . 14
9. Real time: LiveKit, latency, and speech synthesis . . . 16
10. System design, deployment and observability . . . 18
11. Trade-offs and behavioural questions . . . 20
12. Rapid-fire concepts . . . 21
Appendix: formula sheet . . . 23

1. Project overview and framing

The opening questions. Answer in under a minute each; they set up everything else.

Q1 Describe the project in two sentences. EASY

It turns a meeting, recorded or live, into a structured record: a summary, decisions, action items with owners, open questions and per-speaker statistics, where every item cites the transcript lines that support it. It covers transcription (Deepgram Nova-3), diarization (pyannote community-1), structured extraction with an LLM, an optional real-time voice agent (LiveKit + ElevenLabs), and an evaluation suite on the AMI corpus.

> ***Follow-up:*** *What makes it more than a wrapper around an API? The overlap handling, the evidence-grounded schema, and the evaluation, especially the calibrated judge.*

In the repo: `README.md`

Q2 Who is the user, and what does success look like for them? EASY

Someone who missed the meeting, or attended and doesn't want to take notes. Success is that they can act without re-watching: they know what was decided, who owes what by when, and what is still open. That framing drives the rubric dimension **summary_usefulness** and why decisions and action items are first-class schema objects rather than sentences buried in a paragraph.

In the repo: `docs/rubric.md`

Q3 Why does every decision and action item carry evidence ids? MEDIUM

Three reasons. (1) **Auditability**: a UI can jump to the exact moment in the audio. (2) **Automatic grounding**: we can check, without another LLM call, that the cited lines exist and lexically support the item, and drop hallucinations. (3) **Better evaluation**: the judge is shown the cited lines, so it checks faithfulness against evidence instead of guessing. It also forces the extractor to be specific, which itself reduces fabrication.

> ***Follow-up:*** *What if the model cites the wrong id? Grounding catches ids that don't exist and weak lexical support; a wrong-but-plausible id is caught by the judge's faithfulness score.*

In the repo: `summarize/extractor.py: ground()`

Q4 What are the four things this project is meant to prove to a hiring manager? MEDIUM

Model choice (why Nova-3, community-1, which Claude tier where), how overlapping speakers are handled, how a summary with no single right answer is evaluated, and the batch-versus-real-time decision. Each has its own section in `docs/design_decisions.md` with alternatives and the metric that measures it.

In the repo: `docs/design_decisions.md`

Q5 If you had two more weeks, what would you build next? HARD

In order of expected value: (1) actually calibrate the judge with two annotators on ~30 records and publish the QWK table; (2) a speech-separation experiment on SDM audio, gated on the size of the strict vs skip-overlap DER gap; (3) speaker naming, mapping SPEAKER_A to real names from self-introductions or calendar invites, because owners like 'SPEAKER_B' are much less actionable; (4) a small review UI that renders evidence links.

> ***Follow-up:*** *Why calibration first? Every summary-quality claim depends on it; without it the judge numbers are opinions.*

2. Data: the AMI Meeting Corpus

Know the dataset well enough to defend every number you report.

Q6 What is AMI and why is it a good fit? EASY

About 100 hours of real meetings, mostly four participants, recorded with close-talk headsets and far-field microphone arrays, with manual word-level transcripts, speaker labels, and human-written abstractive summaries including decision and action lists. It is the only widely used corpus that has all three references we need: words for WER, speaker timing for DER, and summaries for content evaluation.

In the repo: `docs/data.md`

Q7 What is the difference between the Mix-Headset and Array1-01 streams, and why evaluate both? MEDIUM

Mix-Headset is the sum of the individual close-talk headsets: clean speech, but still one channel with overlap. Array1-01 is a single distant microphone (SDM): reverberation, noise and lower signal level. SDM is closest to a laptop in a meeting room, so it is the realistic number; Mix-Headset isolates the overlap and diarization problem from acoustic difficulty. The gap between them tells you how much is acoustics versus speaker separation.

In the repo: `data/ami.py`

Q8 Which split do you report on and why does it matter? MEDIUM

The standard full-corpus-ASR partition: iterate on dev (and a 3-meeting dev-mini), report on test once. Using the standard partition makes numbers comparable with published work, and touching test only once avoids tuning prompts or thresholds on the data you report. Summary metrics use test-scenario, the test meetings that have abstractive summaries.

> ***Follow-up:*** *Which meetings are in test? EN2002, ES2004, IS1009, TS3003, TS3007, a through d.*

In the repo: `data/ami.py: SPLITS`

Q9 How do you build reference speaker turns for DER from AMI? MEDIUM

From the word annotations: for each speaker, merge consecutive words separated by at most 0.3 s into one turn; overlaps between speakers are preserved. For publication-comparable numbers you should instead load the community-standard RTTM files (BUT's AMI-diarization-setup), because small differences in how turns are built move DER by a point or more.

> ***Follow-up:*** *Why does the merge gap matter? Too small creates many short turns and penalises systems for natural micro-pauses; too large marks pauses as speech, inflating missed speech.*

In the repo: `AMIReference.turns()`

Q10 What are the limitations of AMI's summary references? MEDIUM

Each meeting has one annotator's abstract, and the decision and action lists are sparse. So a system item that is correct but absent from the reference looks like a false positive. That is why matcher precision is reported as a lower bound, why faithfulness is judged against the transcript rather than the reference, and why the judge prompt explicitly says a correct item missing from the reference is not an error.

In the repo: `evals/judge.py`

Q11 How would results on AMI transfer to your company's sales calls? HARD

Partially. AMI is English with many non-native speakers, scenario-driven (designing a remote control), four-person, in-room. Sales calls are two-party, telephony bandwidth, domain jargon, and often two channels. Expect lower DER (two speakers, often separate channels) but different ASR errors. The pipeline transfers; the numbers don't. I'd build a small in-domain eval set of 20-30 calls with the same rubric before claiming anything.

3. Transcription with Deepgram Nova-3

ASR choices, configuration, and failure modes.

Q12 Why Deepgram Nova-3 over self-hosted Whisper? EASY

One model family for batch and streaming, so batch-vs-real-time comparisons isolate the mode, not the model. Word timestamps and confidences, key-term prompting for domain words, multichannel support, and no GPU operations. Whisper is strong on WER but has no native streaming, can hallucinate text on silence, and needs GPU serving. The ASR is behind an adapter that returns `list[Word]`, so swapping vendors is a one-file change.

> ***Follow-up:*** *When would you switch to Whisper? If data can't leave your infrastructure, or if per-minute cost at scale exceeds the cost of running GPUs.*

In the repo: `asr/deepgram_batch.py`

Q13 Why call the REST API directly instead of using the Deepgram SDK? MEDIUM

The Python SDK changed its surface across major versions while the `/v1/listen` HTTP endpoint stayed stable. Calling it directly with `requests` keeps the pipeline version-proof and makes retries and timeouts explicit. The trade-off is losing typed response objects, which the parser replaces.

In the repo: `DeepgramBatchASR`

Q14 What is key-term prompting and when does it help? MEDIUM

Nova-3 accepts `keyterm` parameters that boost recognition of specific words or phrases at inference time, without fine-tuning. It helps on rare domain words that a general model under-recognises (product names, jargon). For AMI we pass the scenario vocabulary (kinetic, scroll wheel, LCD). Risk: over-boosting can cause false insertions of those terms, so it should be validated by comparing WER with and without (`--no-keyterms`).

In the repo: `AMI_KEYTERMS`

Q15 Why enable filler words when you remove them before scoring? MEDIUM

Two different uses. For scoring, fillers are removed from both reference and hypothesis because their annotation is inconsistent and they carry no content. For the transcript itself, keeping 'um' and 'uh' preserves timing and hesitation signals, which help the summarizer tell a tentative proposal from a firm decision.

In the repo: `evals/wer.py`

Q16 What does `smart_format` change and how does it interact with WER? MEDIUM

It formats numbers, dates and punctuation ('twenty five' becomes '25'). That is good for readability but would create artificial errors against AMI's spelled-out references, so normalisation converts digits back to words, maps equivalents (OK/okay), and strips punctuation on both sides before computing WER.

In the repo: `normalize()`

Q17 Your WER is 3 % on Mix-Headset but 25 % on SDM. Where do you look first? HARD

Break errors into substitutions, deletions and insertions. A deletion-heavy profile on SDM usually means quiet or overlapped speakers being dropped; look at deletions inside reference overlap regions versus outside. Substitution-heavy suggests acoustics (reverberation), where beamforming the multi-mic array or a dereverberation front end helps. Then check per-speaker WER: one speaker far from the mic often dominates.

> ***Follow-up:*** *How do you check deletions in overlap? Intersect the alignment's deleted reference words with frames where the reference has 2+ speakers.*

Q18 How do you handle a 2-hour recording? HARD

Pre-recorded ASR handles long files, but for robustness I'd split on silences into chunks of a few minutes with a small overlap and stitch by timestamps, so a single failure doesn't lose everything. Diarization must still run on the whole file (or with consistent embeddings across chunks) or speaker labels won't match across chunks. The summarizer already windows the transcript, so it scales linearly.

4. Diarization and overlapping speech

The section interviewers dig into most.

Q19 What is speaker diarization? EASY

Answering 'who spoke when': segmenting audio into speaker turns and clustering them so the same person gets the same label, without knowing identities in advance. A modern pipeline does local segmentation (including overlapped speech) on short windows, extracts speaker embeddings, and clusters them globally.

Q20 Why pyannote community-1? EASY

Open weights under CC BY 4.0, runs on-prem, improves on the previous open pipeline (3.1) on every AMI condition in its model card (17.0 % DER on AMI IHM and 19.9 % on SDM versus 18.8 % and 22.7 %, no collar, overlap scored), and it returns an exclusive diarization alongside the regular one, which is exactly what word-to-speaker assignment needs.

> ***Follow-up:*** *What would you use if accuracy mattered more than cost? pyannote's hosted precision-2, which the card lists at 15.6 % on AMI SDM; it's a one-line config change.*

In the repo: `diarization/pyannote_diar.py`

Q21 What's the difference between the regular and exclusive diarization outputs, and how do you use each? MEDIUM

The regular output lets turns overlap: two people can be active at the same time. The exclusive output has at most one speaker at any instant. We score DER and detect overlap with the regular view, and assign ASR words to speakers with the exclusive view, because a word appears once in the transcript and must belong to one speaker.

In the repo: `align.assign_speakers()`

Q22 Walk through how a word gets its speaker label. MEDIUM

Find exclusive turns that overlap the word in time and pick the one with the largest overlap. If none overlaps, the word is in a gap where ASR heard speech and diarization didn't: snap it to the nearest turn within 0.5 s of its midpoint. If still nothing, keep Deepgram's own diarization label as a fallback, else UNKNOWN. Separately, check the overlapping view: if two or more speakers cover most of the word, set `overlap=True`. Lookup uses a sorted interval index, so it's O(W log T).

In the repo: `align.py: TurnIndex`

Q23 Why does overlapping speech matter so much for meeting summaries specifically? MEDIUM

Because overlap clusters at the moments that matter: agreement ('yeah, let's do that'), interruptions and decisions. Single-channel ASR tends to transcribe only the dominant speaker, so the other words are lost, and the words that survive can be attributed to the wrong person. That produces wrong owners on action items and unattributed agreements, which are the most damaging errors in a meeting record.

Q24 List every place your system handles overlap. HARD

(1) Diarization keeps the overlapping view. (2) Alignment uses the exclusive view and flags overlapped words. (3) Utterances carry `overlap_ratio` and `co_speakers`. (4) The prompt marks overlapped lines and tells the model attribution is less reliable there. (5) Grounding lowers confidence for items whose evidence is mostly overlapped. (6) Multichannel mode and per-participant tracks remove the problem at the source, with an energy-based cross-talk filter. (7) Measurement: DER strict vs skip-overlap, cpWER vs WER, and per-speaker overlap time.

In the repo: `docs/design_decisions.md section 2`

Q25 How does the cross-talk filter work, and when does it fail? HARD

With one headset per speaker, each headset also picks up neighbours at lower level. For each word on channel c we compare c's mean frame energy during the word to the loudest channel; we keep the word only if c is within 6 dB of the loudest. Bleed is typically well below that. It fails when two people genuinely overlap at similar loudness (both are kept, correctly), when a speaker is very quiet relative to a loud neighbour (their real words can be dropped), and with poor headset placement. The margin should be tuned on dev.

In the repo: `align.crosstalk_filter()`

Q26 Would you add a speech separation model before ASR? HARD

Not by default. Separation can recover the hidden speaker in overlap, but adds latency and GPU cost and can introduce artefacts that increase WER on non-overlapped speech, which is most of the audio. I'd run it as an experiment gated on evidence: if DER strict minus DER skip-overlap is large and cpWER deletions concentrate in overlap regions, separation on overlap segments only is worth testing.

Q27 Should you pass the true number of speakers to pyannote? MEDIUM

Not in the headline number: in production you rarely know it. We run blind with min 2, max 8 and report an oracle-count run separately. The gap between them measures how much error comes from speaker counting. In real time we don't cluster at all: each LiveKit participant is a speaker.

In the repo: `PipelineOptions`

Q28 In real time, how do you know who's speaking without running pyannote? HARD

Track identity: in LiveKit each participant publishes their own microphone track, and we run one STT stream per track, so the speaker is the participant. That's both more accurate and lower latency than online clustering. It breaks when several people share one room microphone; then you fall back to the STT provider's streaming diarization, and accept lower accuracy, or run batch diarization after the meeting.

In the repo: `realtime/agent.py: MultiUserTranscriber`

5. Alignment and the structured record

From words and turns to a schema someone can use.

Q29 How are words grouped into utterances? EASY

Consecutive words from the same speaker are merged while the gap is under 0.8 s and the utterance is shorter than 30 s. Each utterance gets a stable id like `u0042`, which is what evidence refers to. Speakers are relabelled SPEAKER_A, SPEAKER_B in order of first appearance so labels are readable and stable across runs.

In the repo: `align.build_utterances()`

Q30 Why put speaker statistics in the record? MEDIUM

Talk time, turns, words and overlap time per speaker are cheap, objective, and often what managers ask for (participation balance). They also help debugging: a speaker with near-zero talk time often signals a diarization split; an overlap time much higher than others can indicate cross-talk errors.

In the repo: `align.speaker_stats()`

Q31 How is overlap time per speaker computed? MEDIUM

A sweep line over turn start and end events. Between consecutive events we know the set of active speakers; if more than one is active, that interval is added to each of them. It's O(T log T) and exact on the turn boundaries.

In the repo: `_overlap_time_per_speaker()`

Q32 Why define the record schema with Pydantic? MEDIUM

One definition serves four purposes: runtime validation of LLM output, the JSON schema sent to the model as the tool's input schema, the FastAPI response model and OpenAPI docs, and serialization to disk. Field descriptions double as instructions to the model ('never invented' on `due`).

In the repo: `schemas.py`

Q33 What's a design decision in the schema you'd defend? HARD

`due` is a free-text string taken verbatim, never normalised to a date by the model. Normalising 'next meeting' or 'Friday' requires the meeting date and calendar context the model doesn't have, so it would invent precision. Normalisation can be done downstream deterministically when the meeting date is known.

In the repo: `ActionItem`

6. LLM extraction and grounding

How the summary is produced and kept honest.

Q34 Why map-reduce instead of one prompt with the whole transcript? EASY

A 40-minute meeting fits in a large context window, but windowed extraction keeps evidence ids local and accurate, parallelises, and is exactly what the real-time path does incrementally, so batch and real time share one extractor. The reduce step then deduplicates, resolves reversals, and writes the global summary.

> ***Follow-up:*** *When would you do a single pass? Short meetings, where one call is cheaper and loses nothing.*

In the repo: `summarize/extractor.py`

Q35 How do windows avoid splitting a decision in half? MEDIUM

Windows repeat the last 6 utterances of the previous window, so a decision that straddles a boundary is seen whole at least once. Duplicates created by the overlap are merged in reduce.

In the repo: `chunk_utterances()`

Q36 How do you get reliable structured output from the LLM? MEDIUM

Forced tool use: we define one tool whose input schema is the Pydantic JSON schema and set `tool_choice` to that tool, so the model's tool input is the object; there is no free text to parse. Pydantic validates it; on failure we send the validation error back as an error tool result and allow one repair attempt.

In the repo: `summarize/llm.py`

Q37 What does the grounding check do? MEDIUM

For each decision and action item: drop it if none of its evidence ids exist; compute lexical support as the share of the item's content words found in the cited utterances plus three neighbours on each side (loose prefix matching); drop it below 0.15; multiply confidence by 0.85 if most evidence is overlapped. Every drop is logged in `record.warnings`, so silent loss is visible.

> ***Follow-up:*** *Why is lexical support a reasonable proxy? Decisions reuse the vocabulary of the discussion; the threshold is low so paraphrases survive, and the judge handles semantics.*

In the repo: `MeetingExtractor.ground()`

Q38 The grounding threshold trades precision against recall. How did you choose it? HARD

It should be tuned on dev: sweep the threshold, and for each value measure decision recall (matcher) and faithfulness (judge). Pick the knee where faithfulness stops improving and recall starts dropping. The default 0.15 is deliberately permissive because a lexical check should only remove clear fabrications; nuanced faithfulness is the judge's job.

Q39 How do you define a decision versus a proposal? MEDIUM

In the prompt: a decision is something the group settled (agreement, choice, approval, rejection); a proposal nobody accepted is not a decision. Statements are phrased as outcomes. In reduce, a later reversal replaces the earlier decision and the reversal is noted in the rationale. This distinction is the most common source of false-positive decisions, so it's also explicit in the rubric's action-item and coverage anchors.

In the repo: `summarize/prompts.py`

Q40 How would you reduce cost by 5x without losing quality? HARD

Use the cheaper model tier for the map step (it's local extraction) and keep the stronger model for reduce only; cache the system prompt; skip windows with no content (very short or filler-only); and run the judge on a sample rather than every meeting in production monitoring. Validate each change with the same eval set: decision recall and faithfulness must not drop beyond a pre-set tolerance.

Q41 What is the heuristic baseline for? MEDIUM

It's a rule-based extractor that uses cue phrases. It lets the full pipeline run offline in tests and the demo, and it's a known-weak control for the judge: if the judge can't score it clearly below the real extractor, the judge isn't discriminating and its numbers are untrustworthy.

In the repo: `summarize/heuristic.py`

7. Speech metrics: WER, cpWER, DER

Definitions, edge cases, and what each one hides.

Q42 Define WER. EASY

Word error rate = (substitutions + deletions + insertions) / number of reference words, from the minimum edit distance alignment of normalised word sequences. It can exceed 100 % if there are many insertions.

In the repo: `evals/wer.py`

Q43 Why is normalisation essential, and what's the risk? MEDIUM

Without it, formatting differences dominate: '25' vs 'twenty five', 'OK' vs 'okay', punctuation and fillers. The risk is over-normalising and hiding real errors, or normalising asymmetrically. Rules apply identically to reference and hypothesis, are listed in the docs, and should be frozen before comparing systems.

In the repo: `normalize()`

Q44 What is cpWER and why report it alongside WER? MEDIUM

Concatenated minimum-permutation WER: concatenate each speaker's words into one stream, for both reference and hypothesis, find the speaker mapping that minimises total errors (Hungarian algorithm), and compute WER under it. It penalises both recognition errors and words given to the wrong speaker. WER is blind to attribution: on our fixture WER is 3 % while cpWER is 12 % because words in an overlap went to the wrong person.

In the repo: `cpwer()`

Q45 Define DER and its three components. EASY

Diarization error rate = (missed speech + false alarm + speaker confusion) / total reference speech time. Miss: reference speech the system labelled as nobody (or too few speakers). False alarm: system speech where the reference has none (or too many speakers). Confusion: speech attributed to the wrong speaker under the optimal one-to-one mapping between reference and system labels.

In the repo: `evals/der.py`

Q46 How do overlapped regions count in DER? MEDIUM

Per reference speaker. If two people speak for one second and the system outputs one of them correctly, that's one second of correct speech and one second of miss, and the denominator counts two seconds. That's why a system that never predicts overlap has a DER floor equal to the overlapped share of speech.

Q47 What do collar and skip-overlap mean, and which do you report? MEDIUM

A collar ignores a short window around each reference boundary (often 0.25 s), forgiving imprecise boundaries. Skip-overlap excludes overlapped reference frames. We report all three: strict (no collar, overlap scored, comparable to the pyannote card), collar 0.25, and skip-overlap. The difference between strict and skip-overlap is the overlap cost.

> ***Follow-up:*** *Why does strict matter? Collars and skipping overlap can make a weak system look good.*

Q48 How does your DER implementation find the speaker mapping? HARD

Frame-based at 10 ms: build boolean activity matrices for reference and hypothesis speakers, compute the co-occurrence matrix as R times H-transposed, and solve the assignment that maximises total co-occurrence with the Hungarian algorithm (`scipy.optimize.linear_sum_assignment`). Then per frame: miss = max(0, Nref - Nhyp), false alarm = max(0, Nhyp - Nref), confusion = min(Nref, Nhyp) - Ncorrect.

In the repo: `diarization_error_rate()`

Q49 Micro or macro average across meetings? MEDIUM

Micro: sum errors over all meetings divided by total reference words (or speech time). Macro (mean of per-meeting rates) over-weights short meetings. Standard reporting for WER and DER is micro.

In the repo: `evals/runner.py: aggregate()`

Q50 Your DER is fine but users say owners on action items are often wrong. What's going on? HARD

DER weights by time, and action-item commitments are short utterances, often in overlap or right after a speaker change, where boundary errors concentrate. Check cpWER and, more specifically, owner accuracy on action items: compare each action's owner to the reference speaker of its evidence utterance. Fixes: boundary-aware assignment (prefer the turn covering most of the utterance rather than each word), and lower confidence on overlapped evidence, which the record already exposes.

8. Evaluating summaries and calibrating the LLM judge

How to evaluate output that has no single right answer.

Q51 Why not just use ROUGE? EASY

ROUGE measures n-gram overlap with a reference. Many good abstractive summaries share few n-grams with the one AMI reference, and a fluent but wrong summary can score well. We log ROUGE-L only as a collapse detector, never to rank systems.

In the repo: `evals/text_metrics.py`

Q52 Describe your evaluation layers for the record. MEDIUM

(1) Structural: schema valid, evidence ids exist. (2) Grounding: lexical support of each item. (3) Atomic recall and precision against AMI reference decisions and actions via an LLM matcher. (4) A rubric judge with four anchored 1-5 dimensions: faithfulness, decision coverage, action-item quality, usefulness. (5) Pairwise A/B in both orders for comparing versions. (6) ROUGE and compression ratio as sanity checks.

In the repo: `docs/evaluation.md`

Q53 What makes a rubric 'anchored', and why does it matter? MEDIUM

Each score level has a concrete description, for example faithfulness 3 means one substantive unsupported claim or a wrong owner. Annotators and the judge score against the same text. Anchors reduce variance between raters and make disagreements diagnosable: you can tell which anchor was ambiguous.

In the repo: `evals/rubric.py`

Q54 Why does the judge write its analysis before the score? MEDIUM

The schema puts `analysis` before `score`, so the model generates its reasoning first and the score is conditioned on it. Scoring first and justifying after invites rationalisation. It also gives you readable reasons to audit.

In the repo: `DimensionScore`

Q55 What biases do LLM judges have, and how do you mitigate each? HARD

Self-preference: use a different model or tier from the extractor. **Position bias** in pairwise comparisons: run both orders and treat disagreement as a tie. **Verbosity bias**: the rubric says to ignore length unless it hurts usefulness, and usefulness rewards concision. **Leniency / score inflation**: anchors say 5 is rare, and calibration measures mean bias. **Instability**: temperature 0, or three samples with the median and the spread reported. **Reference anchoring**: the prompt says a correct item missing from the reference is not an error.

In the repo: `evals/judge.py`

Q56 How do you calibrate the judge? Give the protocol and the gate. HARD

Two annotators score about 30 records on the rubric independently. Inter-annotator quadratic-weighted kappa is the ceiling. Run the judge on the same records. Per dimension compute QWK, Spearman, exact and within-one agreement, and mean bias (judge minus human). Tune anchors and examples only on a dev half; report on the held-out half. Gate: QWK at least 0.6 and absolute bias at most 0.5 on every dimension.

> ***Follow-up:*** *What if humans only agree at QWK 0.5? Then the rubric is the problem: clarify anchors before blaming the judge; the judge can't beat the human ceiling meaningfully.*

In the repo: `evals/calibration.py`

Q57 Why quadratic-weighted kappa rather than accuracy or Pearson? MEDIUM

Scores are ordinal. Exact accuracy treats 4-vs-5 like 1-vs-5. Pearson ignores systematic offsets and assumes interval data. QWK is chance-corrected and penalises disagreements by squared distance, so near-misses cost little and large misses cost a lot. We still report bias separately, because QWK can be decent with a constant +0.5 offset.

In the repo: `quadratic_weighted_kappa()`

Q58 How is decision recall computed? MEDIUM

An LLM matcher receives the reference decisions and the system decisions and, for each reference item, returns the index of a matching system item or null; recall is the share matched. For precision it marks whether each system item is supported by any reference item; because AMI references are sparse, that precision is a lower bound.

In the repo: `ItemMatcher`

Q59 How would you detect that the judge has drifted after a model update? HARD

Keep the calibration set frozen and re-run the judge whenever its model or prompt changes; compare QWK and bias to the last accepted run. Also keep a few known-bad records (fabricated decision, wrong owner) as canaries that must score low. Pin the judge model version in eval config so drift is only introduced deliberately.

Q60 When do you use pairwise instead of absolute scoring? MEDIUM

When comparing two versions of the system, such as a prompt change. Relative judgments are more reliable than absolute scores, and running both orders cancels position bias. Absolute rubric scores are for tracking quality over time and setting thresholds.

In the repo: `PairwiseJudge`

9. Real time: LiveKit, latency, and speech synthesis

Latency definitions, budgets and trade-offs.

Q61 Define TTFO, TTFW, WPS and TTFA. EASY

TTFO, time to first output: speech onset to the first interim transcript for that segment. **TTFW**, time to first word: LLM request to its first generated token. **WPS**, words per second the LLM streams. **TTFA**, time to first audio: end of the user's turn to the first synthesized audio played, the latency a user actually feels.

In the repo: `evals/latency.py`

Q62 Why does WPS matter if TTFA is already low? MEDIUM

Text-to-speech consumes text at roughly speaking rate, about 2.5-3 words per second. If the LLM streams slower than that, audio stalls mid-sentence after a fast start. WPS should be comfortably above speech rate; we target over three times.

Q63 How is TTFO measured so it isn't distorted by how long someone talks? MEDIUM

Each segment's onset is placed on the stream's own clock: stream start plus the segment's start time from the STT result. TTFO is the wall time of the first interim for that segment minus that onset. Measuring from 'when I started streaming' would mix in speaking duration.

In the repo: `MultiUserTranscriber._run`

Q64 Break down a TTFA budget. MEDIUM

End-of-turn detection (here 300 ms endpointing, or a turn-detector model), plus LLM time to first token, plus time until the first complete sentence, plus TTS time to first byte, plus network and playout. LiveKit reports EOU delay, LLM ttft and TTS ttfb per speech id; we sum them per turn. Target under 1.5 s p50.

In the repo: `agent.py: _wire_metrics`

Q65 What levers reduce TTFA, and what does each cost? MEDIUM

Shorter endpointing (more interruptions of people mid-thought; use a semantic turn detector). A smaller, faster LLM for answers (weaker reasoning; mitigated by grounding answers through a tool that returns the record). Sentence-level streaming from LLM to TTS (needs a sentence splitter that handles abbreviations). Shorter prompts and prompt caching. A faster TTS model (less expressive voice).

In the repo: `docs/realtime.md`

Q66 Why ElevenLabs Eleven v3 Conversational? MEDIUM

It's the expressive v3 voice made for real-time agents, so recaps sound natural rather than robotic. TTS is only on the critical path for spoken answers, not for transcription or notes. ElevenLabs is no longer served through LiveKit Inference, so the agent uses the ElevenLabs plugin with its own key, and the model is configurable with a Flash fallback if a plugin version rejects the id.

In the repo: `config.py: eleven_model`

Q67 How do live notes work, and what's lost compared with batch? HARD

Final transcript segments accumulate; every 45 s or 25 utterances we run the same map-step extractor on the new utterances plus six of context, in a thread so the audio loop isn't blocked, and merge results by text similarity: a restated decision updates the existing one. What's lost: no global reduce, so early decisions that are later reversed may linger, and notes are up to ~45 s stale. At the end, `finalize()` runs the full batch extractor, so the stored record has batch quality.

In the repo: `realtime/live_notes.py`

Q68 Batch or real time: how do you decide? HARD

Default to batch for post-meeting records: cheaper, better diarization (global clustering), and a global reduce, and nobody is waiting. Use real time only when the value is during the meeting: live captions, 'what did we just decide?', catching action items before people leave. Even then, replace the live record with a batch pass at the end. The cost of real time is always-on STT per participant plus repeated LLM updates.

In the repo: `docs/design_decisions.md section 4`

Q69 Why separate the agent's AgentSession from the multi-user transcriber? MEDIUM

AgentSession is built for a conversation with one linked participant: turn detection, interruptions, and replies. A meeting note-taker must transcribe everyone. So a separate component opens one STT stream per participant track for notes, while the AgentSession handles only addressed questions and recaps. The cost is a duplicate STT stream for the linked participant.

In the repo: `realtime/agent.py`

Q70 How do you benchmark real-time latency reproducibly? HARD

Two levels. Component level (`bench_realtime.py`): stream an AMI file to Deepgram at exactly 1x speed paced by the wall clock, then run recap turns through LLM and TTS streaming, recording TTFO, TTFW, WPS and TTFA; same audio, same prompts, so runs compare. System level: run the agent in a LiveKit room and use its metrics events, which include WebRTC and turn detection. Report percentiles (p50, p95), never just means.

In the repo: `scripts/bench_realtime.py`

10. System design, deployment and observability

Production questions.

Q71 Describe the deployment. EASY

Two Docker images. The API image has FastAPI, the batch pipeline and pyannote on CPU torch wheels. The agent image has LiveKit Agents and no torch; it's deployed to LiveKit Cloud with `lk agent create`. They scale on different axes: the API on uploads and GPU minutes, the agent on concurrent rooms.

In the repo: `docs/deployment.md`

Q72 How does the API process long jobs? MEDIUM

Upload returns 202 with an id; processing runs in a background task; the client polls status and fetches the transcript and record. Jobs are stored as JSON on disk, so a restart doesn't lose finished results. For multiple replicas, swap the store for Postgres or Redis and the background task for a queue with GPU workers; both are behind small interfaces.

In the repo: `api/main.py`

Q73 Why cache every stage? MEDIUM

ASR and diarization are the expensive, slow, deterministic stages. Caching them means changing a prompt re-runs only the LLM stages and changing the rubric re-runs only the judge, which makes iteration on dev cheap and keeps eval runs comparable because they share identical upstream outputs.

In the repo: `pipeline.py`

Q74 What do you trace in Arize? MEDIUM

One trace per meeting with spans for ASR, diarization, alignment, extraction (map calls and reduce as LLM spans), and evaluation, using OpenInference attributes. Eval scores are attached as `eval.*` attributes, so you can filter traces by low faithfulness or high DER and open the exact inputs and outputs. Without keys it's a no-op.

In the repo: `observability.py`

Q75 How do you monitor quality in production, where there are no references? MEDIUM

Reference-free signals: grounding drop rate (warnings per record), share of overlapped evidence, average confidence, number of decisions and actions per hour, judge faithfulness on a daily sample, and user feedback (edits to owners or deleted items). Alert on shifts relative to a baseline window, then inspect the traces.

Q76 What are the privacy considerations? HARD

Meeting audio is personal data. pyannote runs in our container, so diarization doesn't send audio to a third party. Deepgram, Anthropic and ElevenLabs receive audio or text: check retention settings and data agreements, and get consent from participants. Delete raw audio after processing if not needed, scope access to records, and keep secrets in environment variables, never in the repo.

In the repo: `docs/deployment.md`

Q77 How would you scale to 10,000 hours per day? HARD

ASR is a managed API, so scale is rate limits and cost; batch requests with backoff. Diarization is the bottleneck: GPU workers pulling from a queue, roughly linear scaling, with audio from object storage. LLM extraction parallelises per window; use the cheaper tier for map. The judge runs on a sample. Store records in a database with evidence ids indexed for the UI.

Q78 What does your test suite cover, and what doesn't it? MEDIUM

41 unit tests with no keys and no network: WER, cpWER and DER edge cases (overlap, collars, relabelling), alignment and overlap flags, cross-talk filtering, RTTM and AMI parsing, Deepgram response parsing, chunking, grounding (dropping fabricated and badly cited items), judge aggregation, pairwise position bias, calibration metrics and gate, live-note merging, and an end-to-end run on a synthetic AMI-format meeting. Not covered: live vendor calls and the LiveKit agent itself, which need keys and are validated with the scripts.

In the repo: `tests/`

11. Trade-offs and behavioural questions

Answer with the situation, the decision, and the evidence.

Q79 Tell me about a trade-off you made deliberately. MEDIUM

Precision over recall in the record. The grounding check drops items with no valid evidence or weak support, accepting that some real decisions might be dropped. A missing decision costs a reader a few seconds of checking; a fabricated decision or a wrong owner can cause real harm and destroys trust in the tool. Drops are logged, so the trade-off is measurable and tunable on dev.

Q80 What was the hardest bug or subtle issue? MEDIUM

Overlap interacting with attribution. On the fixture, WER was 3 % but words from one speaker's overlapped tail were assigned to the person who interrupted. Nothing in WER showed it; cpWER did (12 %). That's why cpWER is a first-class metric and why overlapped evidence lowers confidence.

Q81 What would you not trust in your own results? MEDIUM

Judge scores until calibration is done, precision against AMI's sparse references, and any real-time numbers measured from a single location, since network distance to the vendors dominates tail latency.

Q82 A PM wants 'perfect summaries' by next month. How do you respond? HARD

Agree on what 'perfect' means in measurable terms: for example faithfulness 4.5 or higher on the calibrated judge, decision recall above a threshold, and zero wrong owners on a reviewed sample. Show current numbers and the biggest error sources from the eval report, and propose the one or two changes most likely to move them. Make clear overlap-heavy and far-field audio has a floor that a month won't remove.

Q83 Why did you build an offline demo with fixtures? MEDIUM

So anyone can run the whole pipeline in seconds without keys, CI can test the real alignment, extraction, grounding and metric code on every commit, and a reviewer can see output immediately. The fixture is built to contain the hard case, an overlap, plus realistic ASR errors.

In the repo: `scripts/demo_offline.py`

12. Rapid-fire concepts

Short answers you should be able to give instantly.

Q84 What is an RTTM file? EASY

A plain-text format for diarization: one line per speaker turn with the recording id, start time, duration and speaker label. Standard input to DER scoring tools.

Q85 What is VAD? EASY

Voice activity detection: deciding which frames contain speech. Used to gate STT, detect turn boundaries, and in diarization pipelines. The agent uses Silero VAD.

Q86 What is endpointing? EASY

Deciding that a speaker has finished an utterance, usually after a silence threshold (300 ms here). Shorter means faster responses but more cut-offs.

Q87 Interim vs final transcripts? EASY

Interims are fast, provisional hypotheses that can change; finals are stable after endpointing. Use interims for captions and TTFO, finals for notes and storage.

Q88 What is the Hungarian algorithm used for here? EASY

Optimal one-to-one assignment: reference to hypothesis speakers in DER and in cpWER.

Q89 IHM vs SDM? EASY

Individual headset microphones (close-talk, one per speaker) versus a single distant microphone. SDM is harder: reverberation, noise, overlap.

Q90 What is a speaker embedding? EASY

A fixed-length vector summarising a speaker's voice from a short segment, used to cluster segments by speaker. community-1 uses a WeSpeaker-based embedding.

Q91 What is forced tool use? EASY

Requiring the model to call a specific tool, so its output is the tool's structured input, validated against a JSON schema.

Q92 What is a collar in DER? EASY

A forgiveness window around reference boundaries where errors are not counted, typically 0.25 s.

Q93 What does QWK of 0 mean? EASY

Agreement no better than chance. 1 is perfect agreement; negative is worse than chance.

Q94 What is WebRTC's role? EASY

Low-latency, peer-to-server media transport with jitter buffering and congestion control; LiveKit uses it to carry each participant's audio track to the agent.

Q95 Why report p95 latency, not just the mean? EASY

Users remember the slow responses. The mean hides tails, and real-time systems are judged by their worst common experience.

Appendix: formula sheet

Metric	Definition
WER	$(S + D + I) / N_{ref}$
cpWER	min over speaker permutations π of Σ_s edits(ref_s, $hyp_{\pi(s)}$) / N_{ref}
DER	$(T_{miss} + T_{FA} + T_{conf}) / T_{ref\ speech}$
Per-frame DER terms	miss = max(0, $N_{ref} - N_{hyp}$); FA = max(0, $N_{hyp} - N_{ref}$); conf = min(N_{ref}, N_{hyp}) - $N_{correct}$
QWK	$1 - \Sigma\, w_{ij}\, O_{ij} / \Sigma\, w_{ij}\, E_{ij}$, $w_{ij} = (i - j)^2 / (k - 1)^2$
Item recall / precision	matched reference items / reference items; supported system items / system items
TTFA (per turn)	EOU delay + LLM TTFT + TTS time-to-first-byte
WPS	generated words / ($t_{last\ token} - t_{first\ token}$)
Grounding support	|content words of item found in cited utterances (+/-3)| / |content words of item|

Latency definitions. TTFO: speech onset to first interim transcript. TTFW: LLM request to first token. WPS: streamed words per second (keep well above ~2.5-3 w/s speaking rate). TTFA: end of user turn to first synthesized audio.

Calibration gate. Quadratic-weighted kappa at least 0.6 and absolute mean bias at most 0.5 on every rubric dimension, measured on a held-out half of the human-labelled set.